The WireGuard Protocol

how a DS securely communicate?

Distributed Systems and Computing - *Mid Sweden University*

14/10/2026

Leonardo Barone

Contents

Some history... 4
- ARPANET 5
- TCP/IP 6
- IPSec 7
- PPTP 9
- OpenVPN 10
- OpenVPN 11

The WireGuard Protocol 12
- What is WireGuard? 13
- WireGuard Simplicity 15
- Configuration 16
- WireGuard Security 17
- Cryptokey Routing 19
- Send-Receive flow 20
- Handshake 21
- Performance 22

Contents (ii)

References 23
References 24
..... 25

Some history…

ARPANET

in 1969 ARPA (US Dep. of Defense) developed a method to *link distant computers* → **ARPANET**

- first packet-switching technology
- limitation: computers within the same network

so how can I connect devices across different networks?

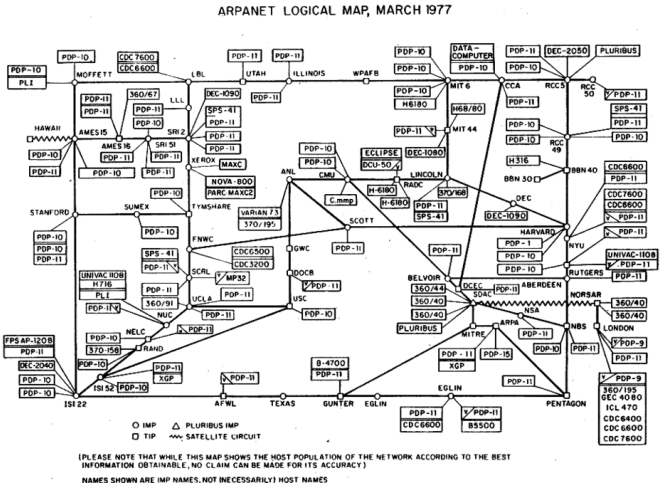


Figure 1: **ARPANET** logical map[1]

[1] https://www.computerhistory.org/collections/catalog/102646704/

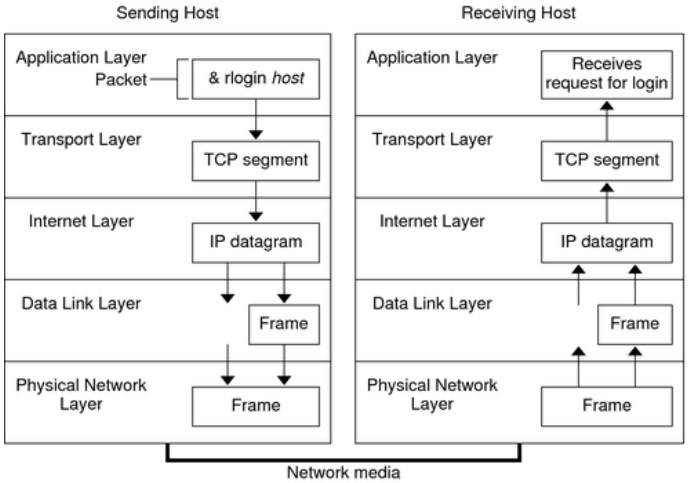


Figure 2: TCP/IP architecture[2]

in the 1970s **TCP/IP** was created, allowing diverse device connections

in 1983 ARPANET switched to TCP/IP
→ birth of **Internet**

something was missing... **ENCRYPTION!**

[2] https://docs.oracle.com/cd/E18752_01/html/816-4554/ipov-29.html

IPSec

- by the mid-1990s
- **IPSec**: ***I***nternet ***P***rotocol ***Sec***urity
- ***suite of protocols*** to provide end2end security by **authenticating** and **encrypting** every IP packet in a data stream
  - (securing connections between devices)

- complex and over-engineered:

> Maybe the code isn't perfect, but I've skimmed it, and compared to the horrors that are OpenVPN and IPSec, it's a work of art.[3]
>
> — Linus Torvalds

[3] while talking about **WireGuard**, we'll get into that later. [1]

IPSec (ii)

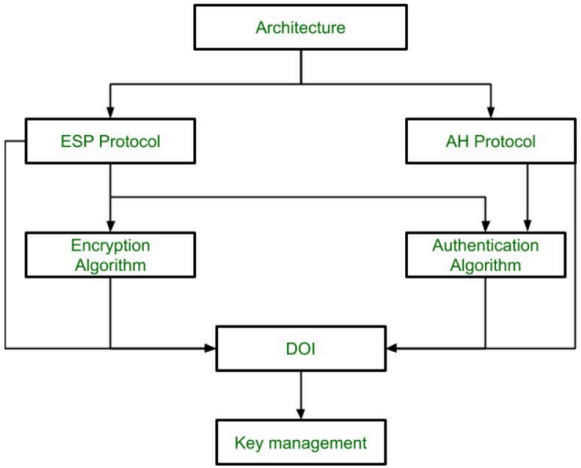


Figure 3: IPSec architecture[4]

- **ESP** : *Encapsulating Security Payload*, provides confidentiality, authentication, and integrity for IP packets. [2]
- **AH** : *Authentication Header*, authenticates the IP packet (including parts of the IP header). [3]
- **IKE** : *Internet Key Exchange*, manages the negotiation and setup of security associations between endpoints, automating the exchange of keys and parameters that AH and ESP use. [4]

[4] https://www.geeksforgeeks.org/computer-networks/ipsec-architecture/

PPTP

- Point-to-Point Tunneling Protocol
- in 1996, the **birth** of **VPNs**
- **virtual tunnel** that allowed data to travel over the public internet as if it were on a private line

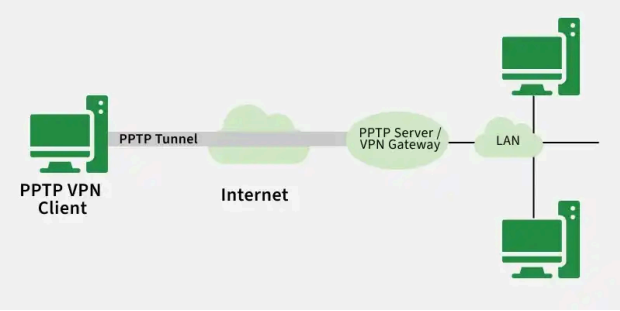


Figure 4: PPTP architecture[5]

too vulnerable! (RC4, MS-CHAP, Snowden revelations..)

[5] https://www.geeksforgeeks.org/computer-networks/pptp-full-form/

OpenVPN

- first major **open-source VPN** protocol (2001)
- the most used between 2005-2020
- both TCP && UDP modes
- SSL/TSL for key exchange (instead of kernel-level IPSec modules) [5]

highly flexible and **secure**, but easy to misconfigure, and its user-space architecture introduces **performance overhead**

OpenVPN

- first major **open-source VPN** protocol (2001)
- the most used between 2005-2020
- both TCP && UDP modes
- SSL/TSL for key exchange (instead of kernel-level IPSec modules) [5]

highly flexible and **secure**, but easy to misconfigure, and its user-space architecture introduces **performance overhead**



The WireGuard Protocol

What is WireGuard?

"WireGuard® is an extremely simple yet fast and modern VPN that utilizes state-of-the-art cryptography. It aims to be faster, simpler, leaner, and more useful than IPsec, while avoiding the massive headache. It intends to be considerably more performant than OpenVPN"[6]

[6] from www.wireguard.com

What is WireGuard? (ii)

again..

> Maybe the code isn't perfect, but I've skimmed it, and compared to the horrors that are OpenVPN and IPSec, it's a work of art.[7]
>
> — Linus Torvalds

[7] [1]

SIMPLE

- < 7000 lines of code[8]
- *fixed* set of cryptographic primitives
- IP packets incapsulated only in **UDP**
- minimal configuration



Figure 6: WireGuard LoC[9]

[8] it was ≈4000 lines
[9] https://nordvpn.com/blog/wireguard-simplicity-efficiency/

Configuration

It's a secure network tunnel at **layer 3**, implemented as a kernel virtual interface

- peers identified by a **public key**
- keys exchanged out of band like SSH[10]

configuration:
interface private key + list of peers (public key, allowed IPs, optional endpoint)

```
$ ip link add dev wg0 type wireguard
$ ip address add dev wg0 10.192.122.3/24
$ wg setconf wg0 configuration-1.conf
$ ip link set wg0 up
$ ping 10.10.10.230
```

[10] [6]

SECURE

- ***fixed* state-of-the-art** cryptography
- default configuration does not store persistent user data
- small codebase → **simple auditing**
- minimal attack surface

WireGuard Security (ii)

- **ChaCha20** and **Poly1305** for AEAD
- **Curve25519** for ECDH
  (with ephemeral keys guarantees PFS)
- **BLAKE2s** for hashing
- **HKDF** for key derivation

all packets sent over ***UDP***

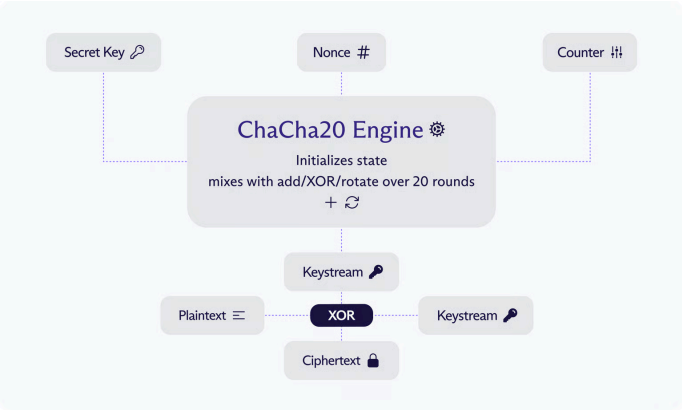


Figure 7: ChaCha20[11]

[11] https://proton.me/learn/encryption/glossary/chacha20

Cryptokey Routing

WireGuard central idea

Peer pub.key	Allowed IPs
`xTIB...p8Dg`	`10.192.122.3/32`, `10.192.124.0/24`
`TrMv...WXX0`	`10.192.122.4/32`, `192.168.0.0/16`
`gN65...z6EA`	`10.10.10.230/32`

- peers identified by their **public key**
  - each key has a list of **allowed IPs**

- for **sending**, the destination IP selects the peer, and that key encrypts the packet
- for **receiving**, after decrypting the packet source IP must map back to the same peer, otherwise dropped

Send-Receive flow

Sending

1. plaintext packet reaches `wg0`
2. destination IP → peer
3. encrypt with the session key and nonce counter (ChaCha20-Poly1305)
4. prepend the WireGuard header
5. send as UDP to the peer's endpoint

Receiving

1. UDP packet arrives on the listen port
2. header → session, check counter, authenticate and decrypt
3. update the peer's endpoint from the outer source IP (for **roaming**)
4. inner source IP must route to *this* peer
5. insert into the `wg0` receive queue

Handshake

how it works?

- **1-RTT** key exchange, Noise **IK** pattern
- either peer can initiate (**symmetric**)
- each side picks a random 32-bit index to tie replies to the session
- **key confirmation:** the responder may send data only after receiving the first data packet from the initiator

what it guarantees?

- to prove identity and create **session keys**
- **replay resistance** (timestamp in the first message)
- **PFS**: fresh ephemeral keys for every session, changed every ≈2min.
- **DoS resistance**: handshake needs Curve25519 multiplication, so with load the responder asks the senders to prove their IP w/ a cookie

Performance

Protocol	Cipher setup	Mbit/s	Ping (ms)
WireGuard	ChaCha20, Poly1305	1,011	0.403
IPsec	AES-256, GCM	881	0.508
OpenVPN	AES-256, HMAC-SHA2-256, UDP	258	1.541

[6]

References

References

[1] L. Torvalds, “Re: [PATCH net-next v6 00/15] WireGuard: fast, modern, secure VPN tunnel.” [Online]. Available: https://lists.openwall.net/netdev/2018/08/02/124

[2] R. J. Atkinson, “IP Encapsulating Security Payload (ESP),” RFC 1827, Aug. 1995. [Online]. Available: https://www.rfc-editor.org/info/rfc1827

[3] R. J. Atkinson, “IP Authentication Header,” RFC 1826, Aug. 1995. [Online]. Available: https://www.rfc-editor.org/info/rfc1826

[4] D. Harkins and D. Carrel, “The Internet Key Exchange (IKE),” RFC 2409, Nov. 1998. [Online]. Available: https://www.rfc-editor.org/info/rfc2409

[5] OpenVPN Community, “Overview of OpenVPN.” Accessed: Oct. 12, 2026. [Online]. Available: https://community.openvpn.net/Pages/OverviewOfOpenvpn

[6] J. A. Donenfeld, “WireGuard: Next Generation Kernel Network Tunnel,” technical report, 2017. [Online]. Available: https://www.wireguard.com/papers/wireguard.pdf

Thank you for the attention :)

Leonardo Barone

Mid Sweden University ∧ University of Bologna

- leonardobarone.com
- github.com/leonardonzio
- *leonardo.barone3@studio.unibo.it*